

The Aether

Specification and Deployment Guide

SubThought Corporation
Version 1.0 — June 2026

The Aether: Specification and Deployment Guide

Copyright © 2013–2026 SubThought Corporation. All Rights Reserved.

No part of this publication may be reproduced, distributed, or transmitted in any form or by any means, without the prior written permission of the publisher, except for brief quotations in reviews and certain other noncommercial uses permitted by copyright law.

THE SOFTWARE IS PROVIDED “AS IS”, WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT.

SubThought, Premise, and GIL are trademarks of SubThought Corporation

Email Premise.AI@Gmail.com for inquiries.

Contents

1 Overview	4
2 The Percept Relation	4
2.1 Slot Semantics	4
3 Percepts by Channel.....	5
3.1 Channel Web.....	5
3.2 Channel FileSystem	5
3.3 Channel Process.....	5
3.4 Channel Message	5
3.5 Channel Browser.....	5
3.6 Channel Memory	5
3.7 Channels Service, System, Embodiment.....	6
4 Common Content Idiom Slots.....	6
5 Actuations	7
6 Cross-Cutting Structure	8
6.1 The Four Tuple Types.....	8
6.2 Actuation → Percept Pairings	8
7 Architecture	9
7.1 Package Layering.....	9
8 Deployment File Set.....	9
9 Adapter Packages.....	9
10 The Theory.....	9
11 Configuration.....	10
12 Framework Asymmetries	10
13 Domain Coverage	10
14 Etymology	11
Index.....	12

3 Percepts by Channel

All percepts from the Aether have `:Modality Aether`. The `:Channel` identifies the domain, `:Data` the format, `:Address` the source, and `:Content` the structured data idiom.

3.1 Channel Web

<code>:Data</code>	<code>:Address</code>	<code>:Content (idiom)</code>	<code>Source</code>
<code>search-result</code>	<code>"https://..."</code>	<code>{:Query ... :Rank 1 :Title ... :Snippet ...}</code>	Both
<code>page-content</code>	<code>"https://..."</code>	<code>{:Title ... :ContentType "text/html" :Size 48000}</code>	Both
<code>api-response</code>	<code>"https://api..."</code>	<code>{:Method GET :Status 200 :Body {...} :Latency 120}</code>	Both

3.2 Channel FileSystem

<code>:Data</code>	<code>:Address</code>	<code>:Content (idiom)</code>	<code>Source</code>
<code>file-content</code>	<code>"file:///home/x"</code>	<code>{:ContentType "text/plain" :Size 2048 :Modified ...}</code>	Both
<code>directory-listing</code>	<code>"file:///home"</code>	<code>{:Entries {{:Name "src" :Type directory} ...}}</code>	Both
<code>file-event</code>	<code>"file:///home/x"</code>	<code>{:EventType modified :Size 2100}</code>	OpenClaw

3.3 Channel Process

<code>:Data</code>	<code>:Address</code>	<code>:Content (idiom)</code>	<code>Source</code>
<code>process-output</code>	<code>nil</code>	<code>{:Command "ls -la" :Stdout ... :ExitCode 0 :Duration 45}</code>	Both
<code>background-output</code>	<code>nil</code>	<code>{:SessionId "bg-42" :Stdout ... :Stderr ""}</code>	OpenClaw

3.4 Channel Message

<code>:Data</code>	<code>:Address</code>	<code>:Content (idiom)</code>	<code>Source</code>
<code>message-received</code>	<code>"telegram://room-7"</code>	<code>{:Sender "alice" :SenderType human :Text "hello"}</code>	Both
<code>agent-reply</code>	<code>"agentverse://agent-3"</code>	<code>{:Protocol agentverse :SessionId "s9" :Body {...}}</code>	Both
<code>webhook-event</code>	<code>"https://hooks..."</code>	<code>{:Source github :EventType push :Body {...}}</code>	OpenClaw

3.5 Channel Browser

<code>:Data</code>	<code>:Address</code>	<code>:Content (idiom)</code>	<code>Source</code>
<code>browser-content</code>	<code>"https://..."</code>	<code>{:Title ... :Selector "article" :ContentType text}</code>	OpenClaw/Port42
<code>media</code>	<code>"file:///frame.png"</code>	<code>{:Url ... :Width 1280 :Height 800 :Format png}</code>	OpenClaw/Port42
<code>accessibility-tree</code>	<code>"https://..."</code>	<code>{:NodeCount 340 :Tree {...}}</code>	OpenClaw

3.6 Channel Memory

:Data	:Address	:Content (idiom)	Source
memory-result	nil	{:Query ... :StoreType embedding :Results {{:Score 0.87} ...}}	Both
knowledge-result	nil	{:Query ... :Results {{:Reifier 12345 :Confidence 0.9} ...} :InferenceTrail {...}}	OmegaClaw

3.7 Channels Service, System, Embodiment

Service: service-resource, financial-data. System: schedule-event, connection-status. Embodiment: media, sensor-reading, motor-state, action-result, transcription, clipboard. See the Aether.md companion file for complete idiom definitions.

4 Common Content Idiom Slots

Slot	Type	Count	Notes
:ContentType / :Format	literal	22	Describes the kind of data
:Size / :Width / :Height / :Duration	integer	15	Dimensional measure
:Source / :Service / :Sender / :DeviceId	literal	16	Who or what produced this data
:Title	string	5	Human-readable label
:SessionId / :ThreadId / :JobId	string	5	Session context
:Status / :ExitCode	literal/int	4	Outcome indicator
:Confidence / :Score	real	3	Certainty measure
:InferenceTrail	list	1	Reasoning provenance (OmegaClaw)

5 Actuations

Actuations are dispatched as ATTEMPT structures. The :Action slot names the literal. The :Parameters slot carries an idiom.

```
[ATTEMPT :Action exec :Parameters {:Command "ls -la" :Timeout 30000} :Token ?tok :By
\@m{...}]
```

The Aether defines 43 actuation literals across 10 domains.

Domain	Count	Literals
Shell Execution	4	exec, exec-background, process-kill, process-write
File System	6	file-create, file-overwrite, file-edit, file-patch, file-delete, file-move
Web Interaction	7	web-search, web-fetch, browser-navigate, browser-click, browser-type, browser-execute, browser-capture
Communication	5	message-send, agent-send, agent-spawn, notify, email-send
Memory	6	memory-store, memory-delete, knowledge-assert, knowledge-retract, nal-infer, pln-infer
Content Generation	3	generate-text, generate-image, render-ui
Automation	2	cron-schedule, cron-cancel
External Services	1	rest-call
Device / Embodiment	7	drive, gimbal, joint, light, speak, clipboard-write, automation-run
Capture	2	capture-screen, capture-camera

6 Cross-Cutting Structure

6.1 The Four Tuple Types

```
PERCEPT: [PERCEPT :Modality Aether :Channel C :Address A :Data D :Content {...}
:Moment M :Token T]
ATTEMPT: [ATTEMPT :Action A :Parameters {...} :Token T :By M]
RESULT: [RESULT :Action A :Status S :Reason R :Moment M :Token T]
URGE: [URGE :Need N :Source S :Delta D :Moment M :Token T]
```

The mind learns what its body did through perception. The result of an ATTEMPT comes back as a RESULT sent to the Perceiver — not the Executor. This is proprioception.

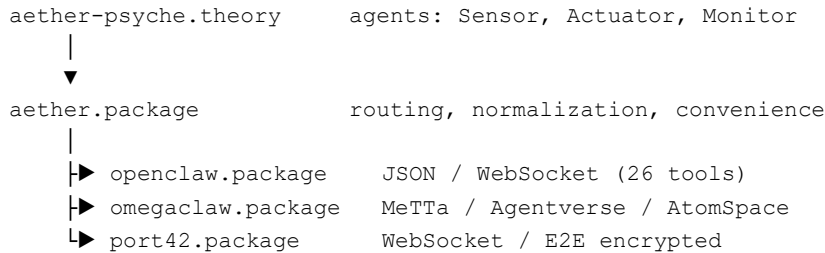
6.2 Actuation → Percept Pairings

:Action	Produces :Data
exec, exec-background	process-output
file-create through file-patch	file-event
web-search	search-result
web-fetch	page-content
browser-navigate, click, type	browser-content or media
browser-capture	media
message-send, agent-send	message-received or agent-reply
memory-store	memory-result
knowledge-assert	knowledge-result
nal-infer, pln-infer	knowledge-result (with :InferenceTrail)
rest-call	api-response
capture-screen, capture-camera	media
cron-schedule	schedule-event (future)
drive-command, joint-command	action-result, motor-state

7 Architecture

The Aether mediates between the mind (Perceiver/Executor) and three adapter frameworks (OpenClaw, OmegaClaw, Port42). The theory's agents call the `aether.package`, which routes to the correct adapter package, which translates to tell/ask calls over TCP, HTTPS, or pipes.

7.1 Package Layering



8 Deployment File Set

File	Category	Description
<code>openclaw.package</code>	Adapter	OpenClaw protocol: 30+ functions, generic dispatch
<code>omegaclaw.package</code>	Adapter	OmegaClaw protocol: 20+ functions, NAL/PLN, AtomSpace
<code>port42.package</code>	Adapter	Port42 protocol: 20+ functions, browser, audio, camera
<code>aether.package</code>	Unified Layer	Routing, normalization, convenience functions
<code>aether-psyche.theory</code>	Theory	Three agents: Sensor, Actuator, Monitor
<code>Aether.daicho</code>	Configuration	Adapter URLs, routing, polling, urge thresholds
<code>aether.suites</code>	Tests	9 suites: lifecycle through full end-to-end

9 Adapter Packages

`openclaw.package` wraps OpenClaw's tool groups (runtime, fs, web, ui, sessions, memory, messaging, automation, clawbody) into 30+ functions prefixed `oc-`, plus a generic `oc-dispatch`.

`omegaclaw.package` wraps OmegaClaw's skill categories (shell, agentverse, atomspace, reasoning, channels, memory) into 20+ functions prefixed `omega-`, plus `omega-dispatch`.

`port42.package` wraps Port42's bridge API domains (browser, screen, camera, audio, clipboard, automation, notify, rest) into 20+ functions prefixed `p42-`, plus `p42-dispatch`.

10 The Theory

`aether-psyche.theory` defines three agents. Sensor polls channels at configured intervals and wraps raw data as PERCEPT tuples. Actuator listens for ATTEMPT tuples from the Executor and dispatches through `aether-act`. Monitor checks homeostatic conditions (connectivity, inbox backlog, disk usage) and emits URGE tuples when thresholds are crossed.

On startup, the theory registers the Aether device with the Registrar, connects all adapters marked `:AutoConnect` yes, and starts the three agents.

11 Configuration

Section	Contents
[Psyche]	Agent URL, port, delay, modality literal
[OpenClaw]	URL, auto-connect, timeout, channels, capabilities
[OmegaClaw]	URL, auto-connect, timeout, channels, capabilities
[Port42]	URL, auto-connect, timeout, channels, capabilities
[Routing]	Per-channel adapter priority lists
[Polling]	Per-channel polling intervals in Moments
[Urges]	Homeostatic need definitions with thresholds

12 Framework Asymmetries

OmegaClaw has, OpenClaw lacks:

Formal symbolic reasoning (NAL, PLN) with auditable proof trails. Self-modifying control loop (MeTTa runtime introspection). Three-tier memory with typed hypergraph knowledge store (AtomSpace). Continuous autonomous execution loop.

OpenClaw has, OmegaClaw lacks:

Full browser automation (Playwright-based, multi-profile). 25+ messaging channel integrations. 5,400+ community skills via ClawHub and MCP. Structured file editing tools. Background process management with PTY. Cron-based scheduling. ROS 2 embodiment bridge (ClawBody).

13 Domain Coverage

Domain	:Channel	Percept :Data	Actuations	Source
Local Compute	FileSystem, Process	file-content, process-output, etc.	exec through file-move	Both
Web	Web	search-result, page-content, api-response	web-search, web-fetch, rest-call	Both
Communication	Message	message-received, agent-reply, webhook	message-send through email-send	Both
Memory	Memory	memory-result, knowledge-result	memory-store through pln-infer	Both
Browser	Browser	browser-content, media, a11y-tree	browser-navigate through capture	OC/P42
External	Service	service-resource, financial-data	rest-call	Both
Embodiment	Embodiment	media, sensor, motor, action, etc.	drive through capture-camera	OC/P42
Automation	System	schedule-event, connection-status	cron-schedule, cron-cancel	OpenClaw

14 Etymology

Aether (Greek Αἰθήρ, Aithēr) — from the verb αἶθω (aithō), “to burn, to ignite.” Originally the bright upper atmosphere that the gods breathed. Aristotle elevated it to the fifth element — the quinta essentia. In modern physics, the luminiferous aether was the medium through which electromagnetic waves propagated. Every era of meaning reinforces the same core idea: Aether is the medium through which things propagate. The internet is exactly this.

Index

- Aether — see §1, §17
- Actuations — see §5
- Adapter packages — see §9
- Agent (Actuator) — see §10
- Agent (Monitor) — see §10
- Agent (Sensor) — see §10
- AtomSpace — see §3.6, §9, §12
- ATTEMPT — see §5, §6
- Browser (channel) — see §3.5
- Channel — see §2, §3
- ClawBody — see §3.9, §12
- :Content (idiom) — see §2, §4
- Configuration — see §11
- :Data (format) — see §2
- Deployment — see §8
- Embodiment (channel) — see §3.9
- Expanse — see §14
- FileSystem (channel) — see §3.2
- GIL — see §1
- Idiom — see §2, §4
- Knowledge graph — see §3.6
- Memory (channel) — see §3.6
- Message (channel) — see §3.4
- :Modality — see §2
- Monitor agent — see §10
- NAL — see §5, §9, §12
- OmegaClaw — see §9, §12
- OpenClaw — see §9, §12
- PERCEPT — see §2, §6
- PLN — see §5, §9, §12
- Port42 — see §9
- Premise — see §1
- Process (channel) — see §3.3
- Psyche — see §1
- RESULT — see §6
- Routing — see §9, §11
- Sensor agent — see §10
- Service (channel) — see §3.7
- System (channel) — see §3.8
- Test suites — see §8
- URGE — see §6, §10
- Web (channel) — see §3.1

